

Salient Object Detection Using a CNN EncoderDecoder Model

1. Introduction

Salient Object Detection is the task of identifying the most visually important object or region in an image. Instead of detecting and classifying every object, the goal is to produce a saliency mask that highlights the dominant object or area that naturally attracts human attention.

In this project, a complete Salient Object Detection pipeline was implemented from scratch using Python and PyTorch. The system takes an RGB image as input and outputs a one-channel binary saliency mask. The project includes dataset preparation, preprocessing, model design, training, evaluation, visualizations, experiments, and a simple demo application.

The main goal of this project was not only to train a working model, but also to understand the full deep learning workflow behind image segmentation-style tasks.

2. Dataset

The ECSSD dataset was used for this project. ECSSD contains natural images and their corresponding ground-truth saliency masks. Each image has a manually annotated mask that shows the visually important object region.

The dataset was organized into two main folders:

- Images
- Ground-truth masks

Each image was paired with its corresponding mask. The dataset contained 1000 images and 1000 masks.

3. Data Preprocessing

Before training the model, the images and masks were preprocessed.

The preprocessing steps included:

- Loading RGB images
- Loading grayscale saliency masks
- Resizing all images and masks to 128x128 pixels
- Normalizing image pixel values to the range 0-1
- Converting masks into binary format
- Splitting the dataset into training, validation, and test sets

The dataset split was:

- Training set: 70%
- Validation set: 15%
- Test set: 15%

Basic data augmentation was applied only to the training set. The augmentations included horizontal flipping and brightness variation. These augmentations were used to make the model more robust and reduce overfitting.

4. Baseline Model Architecture

The baseline model was implemented as a convolutional encoder-decoder network from scratch.

The encoder part was responsible for extracting visual features from the input image. It used multiple convolutional layers followed by ReLU activation and max pooling.

The decoder part was responsible for reconstructing the saliency mask from the encoded features. It used transposed convolution layers for upsampling.

The model structure followed this general design:

- Input: RGB image with shape 3 x 128 x 128
- Encoder: Conv2D, ReLU, MaxPooling
- Decoder: ConvTranspose2D, ReLU
- Output: 1-channel mask with Sigmoid activation

The Sigmoid activation was used because the task is a binary pixel-level prediction problem, where each pixel belongs either to the salient object or the background.

5. Loss Function

The model was trained using a custom loss function:

Binary Cross-Entropy + $0.5 * (1 - \text{IoU})$

Binary Cross-Entropy helps the model learn pixel-level classification between foreground and background. The IoU component helps the model improve the overlap between the predicted mask and the ground-truth mask.

Using both losses together is useful because saliency detection needs both pixel-level accuracy and good region-level overlap.

6. Training Process

The model was trained using the Adam optimizer with a learning rate of 0.001. The training process included a full custom training loop with:

- Forward pass
- Loss calculation
- Backward pass
- Optimizer update
- Validation after each epoch
- Metric logging
- Saving the best model
- Saving training checkpoints

The model was trained for up to 20 epochs. Early stopping logic was included to stop training when the validation loss stopped improving.

Checkpoints were saved during training so that the model could resume training if the process stopped unexpectedly.

7. Evaluation Metrics

The model was evaluated using the following metrics:

- Intersection over Union
- Precision
- Recall
- F1-score
- Inference time per image

Intersection over Union measures how much the predicted mask overlaps with the ground-truth mask.

Precision measures how many predicted salient pixels were actually correct.

Recall measures how many real salient pixels were successfully detected.

F1-score balances precision and recall into one score.

Inference time measures how long the model takes to generate a saliency mask for one image.

8. Baseline Results

The baseline model achieved the following test results:

Model	IoU	Precision	Recall	F1-score	Inference Time
Baseline CNN	0.3632	0.6096	0.4779	0.5280	0.0015 sec

The baseline model was able to detect general salient object regions, but the predicted masks were sometimes incomplete, noisy, or blocky. This is expected because the baseline model is a simple CNN encoder-decoder trained from scratch.

The visual results showed that the model learned the general idea of saliency, but it struggled with clean object boundaries and complex backgrounds.

9. Improved Model Experiment

After the baseline model was completed, an improved model was created.

The improved model added two architectural changes:

- Batch Normalization
- Dropout

Batch Normalization was added to stabilize training and help the model learn better feature representations.

Dropout was added to reduce overfitting by randomly dropping some activations during training.

The improved model kept the same general encoder-decoder structure, but these changes made the training process more stable and improved performance.

10. Improved Model Results

The improved model achieved the following test results:

Model	IoU	Precision	Recall	F1-score	Inference Time
Baseline CNN	0.3632	0.6096	0.4779	0.5280	0.0015 sec
Improved CNN	0.3891	0.6012	0.5344	0.5558	0.0016 sec

The improved CNN achieved better IoU, Recall, and F1-score compared to the baseline model.

IoU increased from 0.3632 to 0.3891, which shows better overlap between the predicted masks and the ground-truth masks.

Recall increased from 0.4779 to 0.5344, meaning the improved model captured more of the salient object regions.

Precision decreased slightly from 0.6096 to 0.6012. This means the improved model predicted more object pixels, but it also included slightly more background pixels.

Overall, the F1-score improved from 0.5280 to 0.5558, so the improved model performed better overall.

11. Visual Analysis

The visualizations showed the input image, ground-truth mask, predicted mask, and overlay.

The baseline model produced masks that roughly focused on the main object, but the predictions were often fragmented and missed parts of the object.

The improved model detected larger parts of the salient objects and had better overall coverage. However, it sometimes included extra background regions. This matches the metric results, where recall improved but precision slightly decreased.

The visual analysis helped confirm that the improved model was better at detecting the full salient object area, even though it still had limitations.

12. Demo Application

A Gradio demo application was created for live testing.

The demo allows the user to upload any image and returns:

- The resized input image
- The predicted saliency mask
- The overlay visualization
- The inference time

This makes the model easier to present and test during the final review.

13. Limitations

The model has several limitations.

First, the input image size was only 128x128 pixels. This made training faster, but it reduced fine details and object boundaries.

Second, the model was trained from scratch without using pre-trained models. This was required by the project goal, but it also means the model had to learn all visual features from a relatively small dataset.

Third, the predicted masks were sometimes blocky or noisy. This happened especially when the image had complex backgrounds or multiple objects.

Fourth, the model sometimes detected extra background areas as salient object regions.

14. Future Improvements

The model could be improved in several ways:

- Use a larger input size such as 224x224
- Add skip connections between encoder and decoder layers
- Use a deeper CNN architecture
- Add more data augmentation
- Train for more epochs
- Tune the loss function weights
- Apply post-processing to clean predicted masks

Skip connections would likely help the model recover more spatial details and produce cleaner masks

15. Conclusion

This project successfully implemented a complete Salient Object Detection system from scratch.

The full pipeline included dataset preparation, preprocessing, CNN model design, custom loss function, training loop, validation, checkpoint saving, evaluation, visualization, experimentation, and a live demo application.

The baseline model learned to detect general salient object regions. The improved model, which added Batch Normalization and Dropout, achieved better IoU, Recall, and F1-score.

The final results show that even a simple CNN encoder-decoder model can learn useful saliency detection patterns, but better architecture and training strategies are needed for cleaner and more accurate masks.